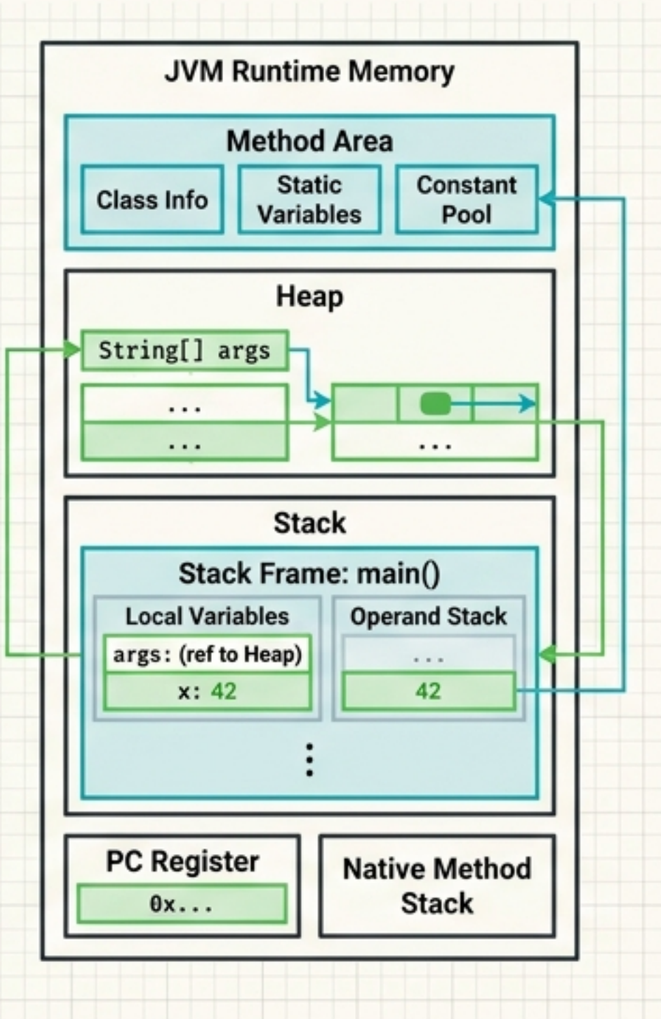
The Core Java Blueprint

A Visual Guide to Syntax, Memory, and JVM Architecture

```java
public class HelloWorld {
  public static void main(String[] args) {
     int x = 42;
     System.out.println(x);
  }
}
```

```
CA FE BA BE 00 00 00 34 00 13 0A 00 03 00
10 07 00 11 07 00 12 01 00 06 3C 69 6E 69
74 3E 01 00 03 28 29 56 01 00 04 43 6F 64
65 01 00 0F 4C 69 6E 65 4E 75 6D 62 65 72
54 61 62 6C 65 01 00 04 6D 61 69 6E 01 00
16 28 5B 4C 6A 61 76 61 2F 6C 61 6E 67 2F
53 74 72 69 6E 67 3B 29 56 01 00 0A 53 6F
72 72 65 46 69 6C 65 01 00 0F 48 65 6C 6C
6F 57 6F 72 6C 64 2E 6A 61 76 61 0C 00 07
07 00 08 07 00 13 0C 00 14 00 15 01 00 10
6A 76 61 2F 6C 61 6E 2F 4F 62 6A 65 63 74
01 00 10 6A 61 6E 67 2F 53 79 73 74 65 6D
01 00 03 6F 75 74 01 00 15 4C 6A 61 76 61
69 6F 2F 50 72 69 6E 74 53 74 72 65 61 6D
3B 53 74 72 65 61 6D 01 00 07 70 72 69 6E
74 6C 6E 6E 01 00 04 28 49 29 56 00 21 00
00 02 00 03 00 00 00 00 00 05 2A B7 00 01
B1 00 00 00 01 00 0A 00 00 00 06 00 00 0B
0B 0C 0C 0C 01 00 09 09 00 00 00 25 00 02
00 01 00 00 00 09 10 2A 3C B2 00 04 1B B6
06 00 05 B1 00 00 00 01 00 0A 00 00 00 0A
00 02 00 00 00 03 00 08 00 04 00 01 00 0D
00 00 00 02 00 0E
```



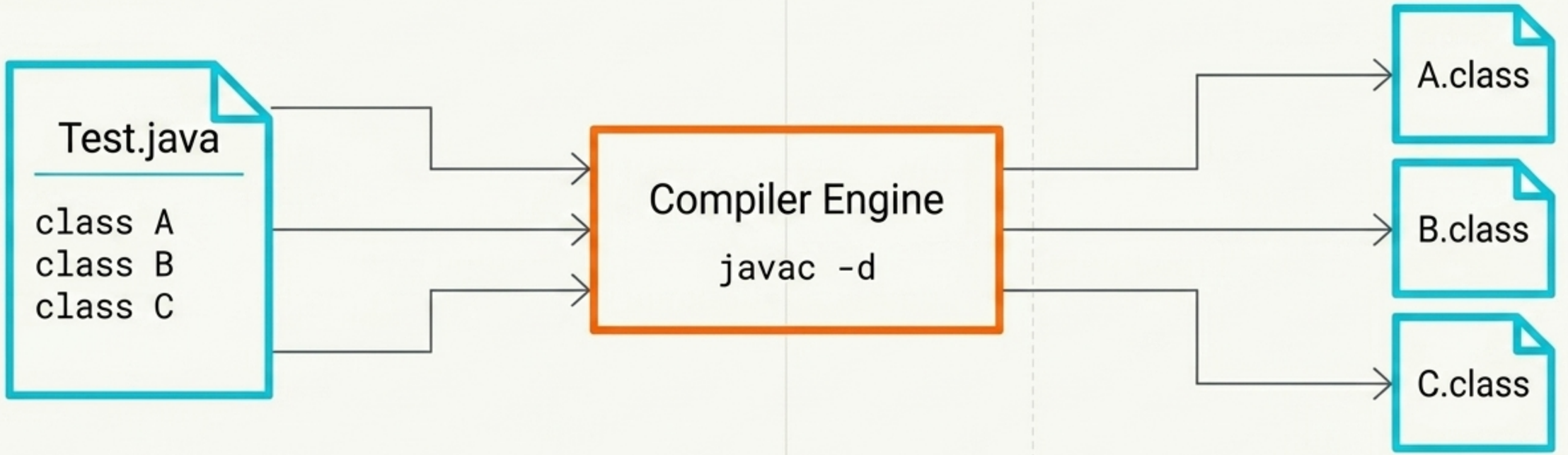
Naming Conventions Matrix

Element	Rule	Example
Classes	PascalCase	`class Employee` `class HelloWorld`
Methods & Variables	camelCase	`double performanceIndex` `void calculateSalary()`
Constants	ALL_CAPS	`final double PI = 3.14159;`

Identifier Rules

- **Allowed Starters:** Letters (`a-z`, `A-Z`), Dollar Sign (`$`), Underscore (`_`)
- **Prohibited:** Cannot start with a number. Generates immediate compile-time error.
- **Keywords:** Cannot use reserved words (e.g., `true`, `if`, `while`).

Java Source File Compilation & Naming Rules

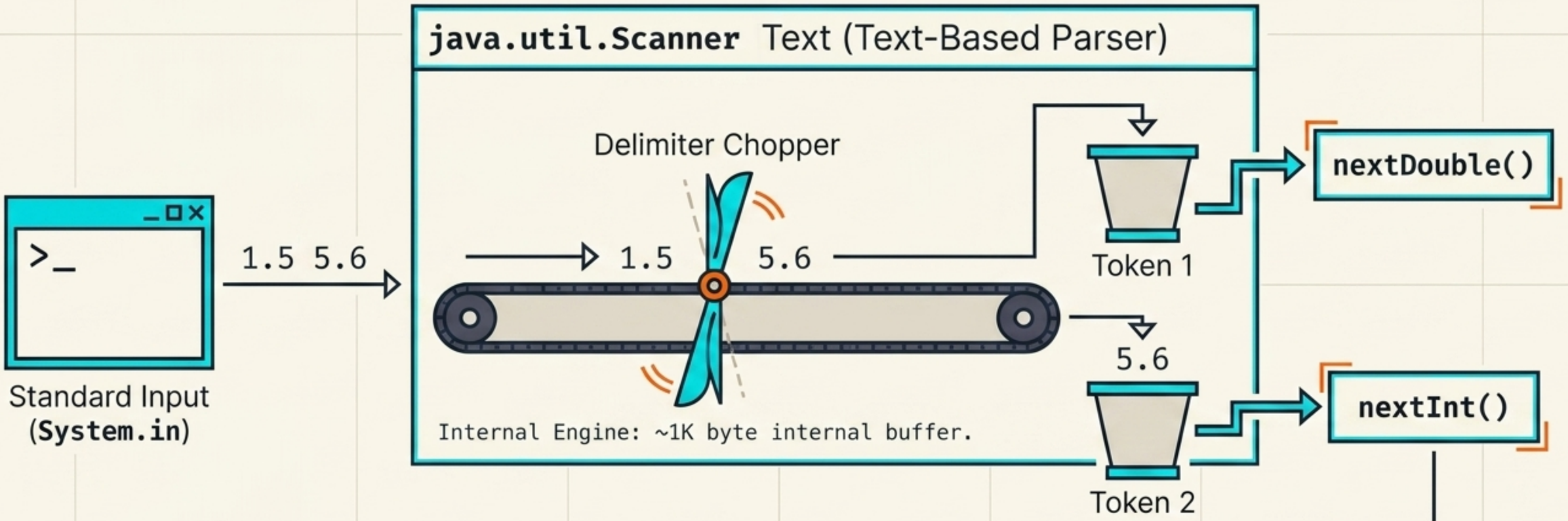


The Strict Naming Rule

Rule: A source file can contain multiple default classes, but a maximum of **ONE** public class.

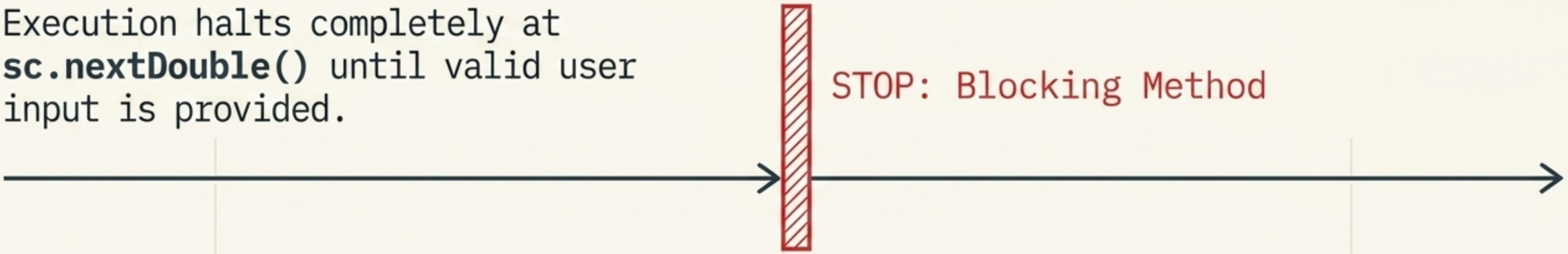
Mandate: The name of the public class **MUST** perfectly match the source file name (e.g., `public class Test` must reside in `Test.java`), otherwise the compiler throws an error.

java.util.Scanner Anatomy: Text-Based Parsing Pipeline

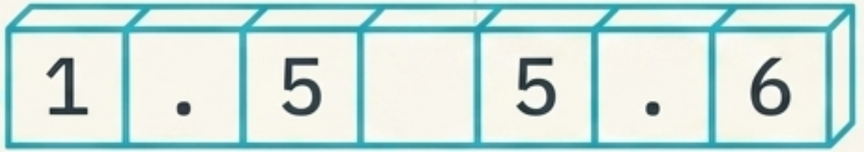


If the string token cannot map to the requested type, it triggers an `InputMismatchException`.

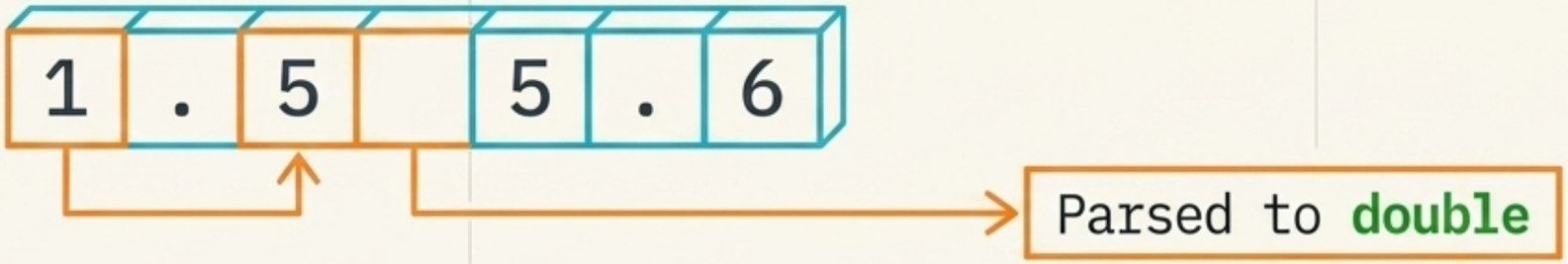
Execution halts completely at **sc.nextDouble()** until valid user input is provided.



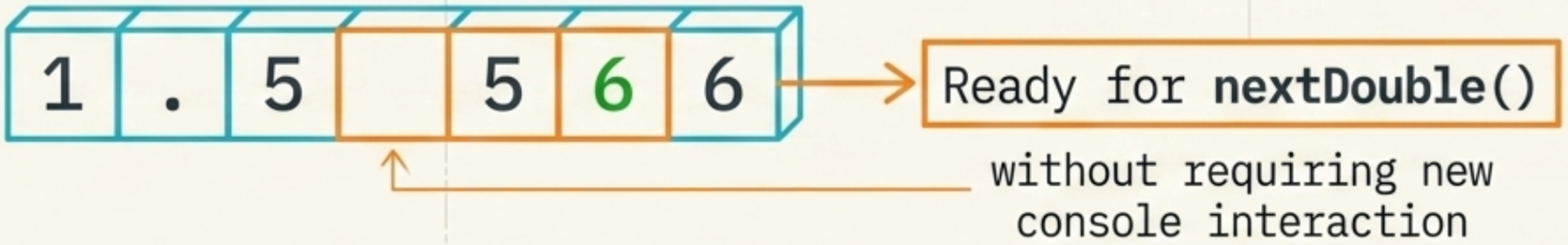
User inputs "**1.5 5.6** [ENTER]"



Pointer reads **1.5**



Execution resumes



Java Data Types

Primitive Types (Value-Holding)

Store raw data values directly in memory.

Non-Numeric

boolean
(true/false)

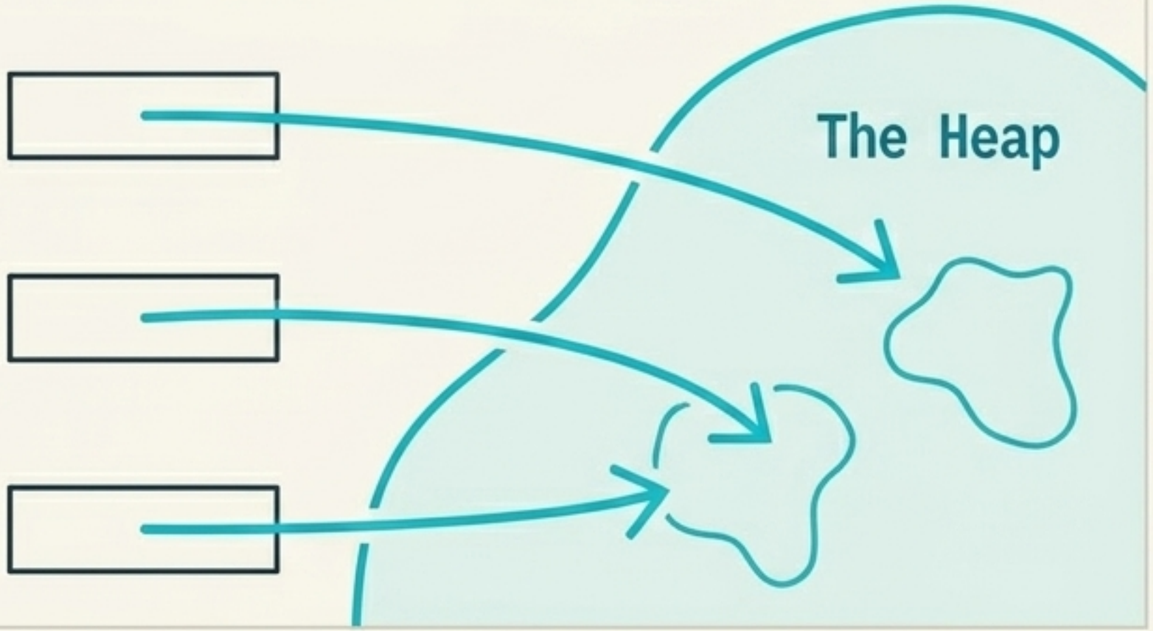
Numeric

byte, short, int, long, char, float, double

Reference Types (Address-Holding)

Do not store the object's data. They store a pointer/address to an object dynamically created at runtime.

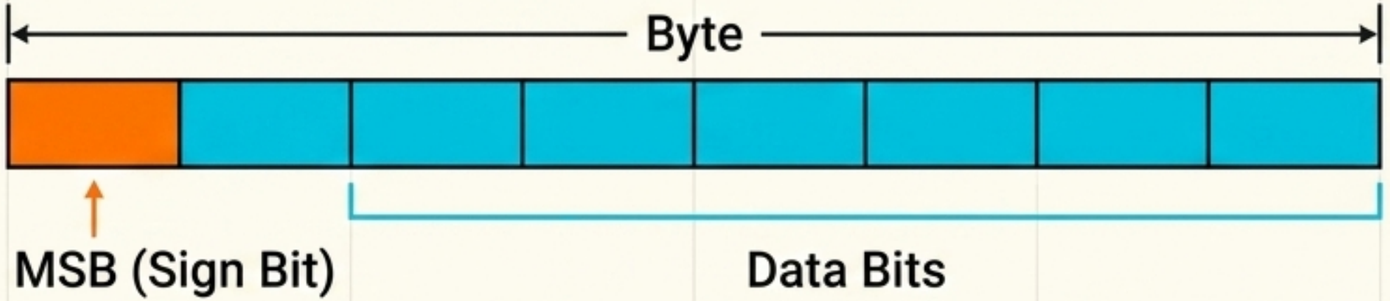
Arrays, Classes, Interfaces



The Primitive Spec Matrix

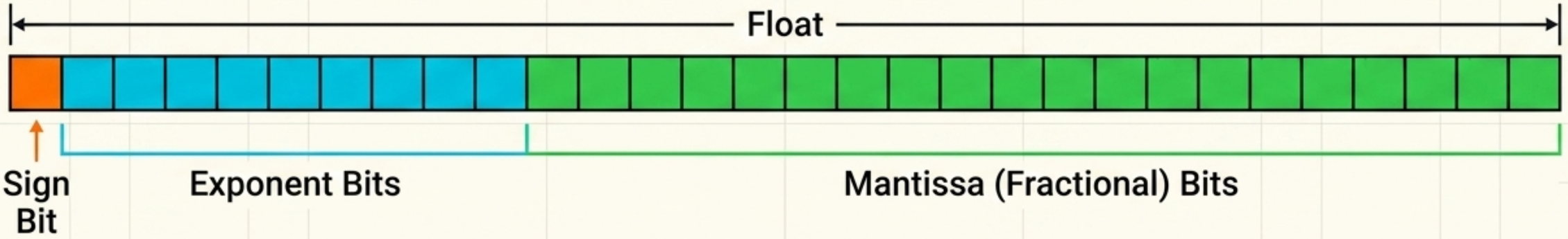
Type	Size	Range Formula / Description
boolean	JVM Dependent	Holds true or false
char	2 Bytes (Unsigned)	0 to 65,535. Sized to support Universal Character Encoding (Unicode).
byte	1 Byte	-128 to 127
short	2 Bytes	-32,768 to 32,767
int	4 Bytes	-(2^31) to (2^31)-1
long	8 Bytes	-(2^63) to (2^63)-1
float	4 Bytes	Single Precision
double	8 Bytes	Double Precision

Integral Types: Two's Complement



`0` = positive, `1` = negative. Negative numbers are stored by flipping the bits (1's complement) and adding 1.

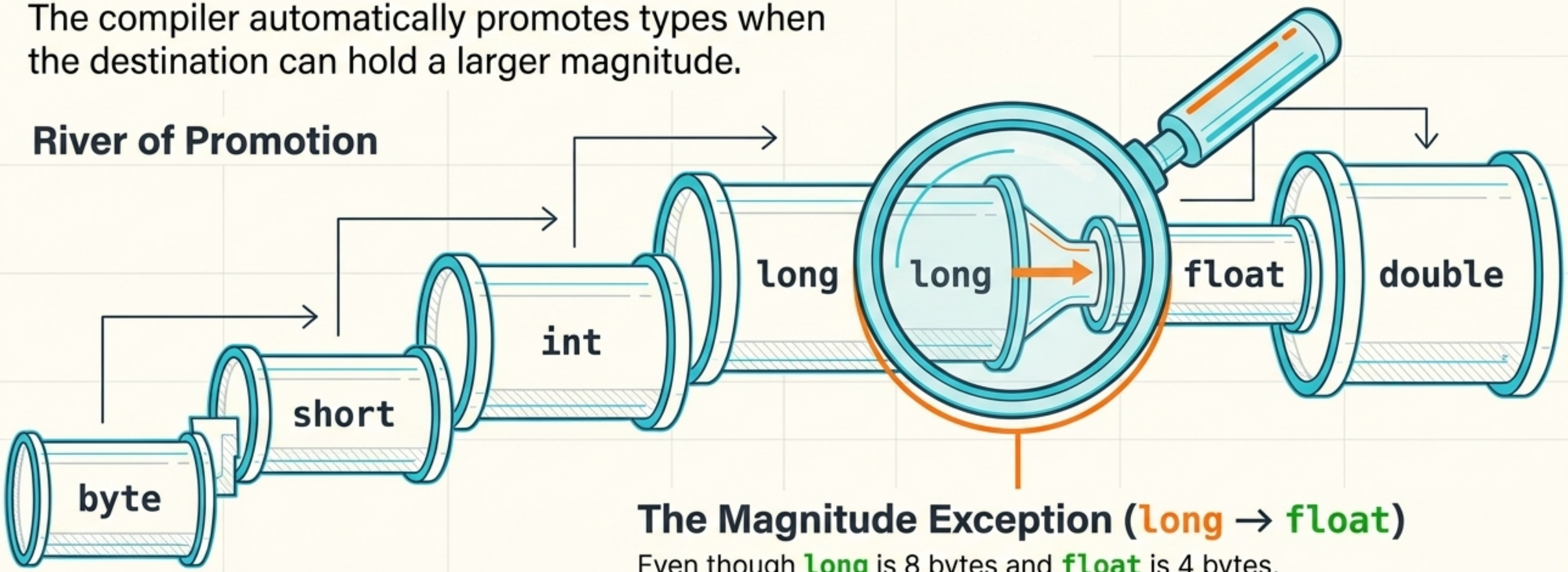
Floating-Point Types: IEEE 754



Double (64 bits) extends this layout using 1 Sign Bit, 11 Exponent Bits, and 52 Mantissa Bits.

Widening (Implicit) Conversion

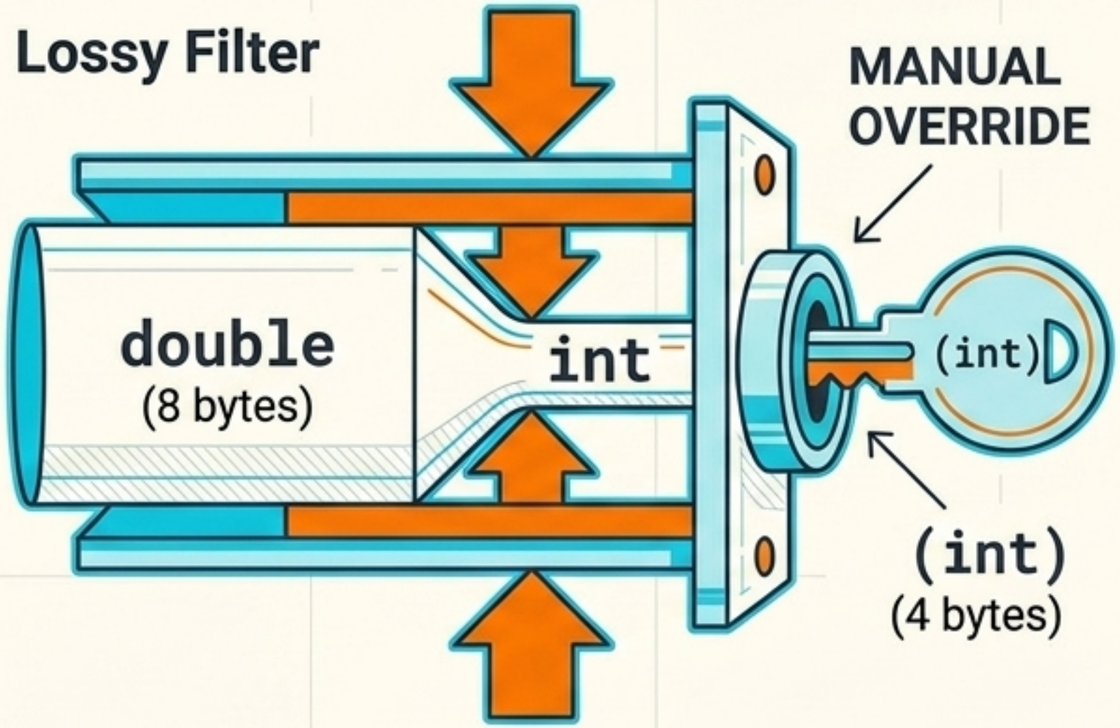
The compiler automatically promotes types when the destination can hold a larger magnitude.



The Magnitude Exception (`long` → `float`)

Even though `long` is 8 bytes and `float` is 4 bytes, automatic promotion is allowed. The 8 Exponent bits in a **float** allow it to store a significantly wider range of values (magnitude) than a 64-bit integer.

Narrowing (Explicit) Conversion



Requires a manual typecast (e.g., `byte b = (byte) myInt;`). Prevents silent loss of precision.

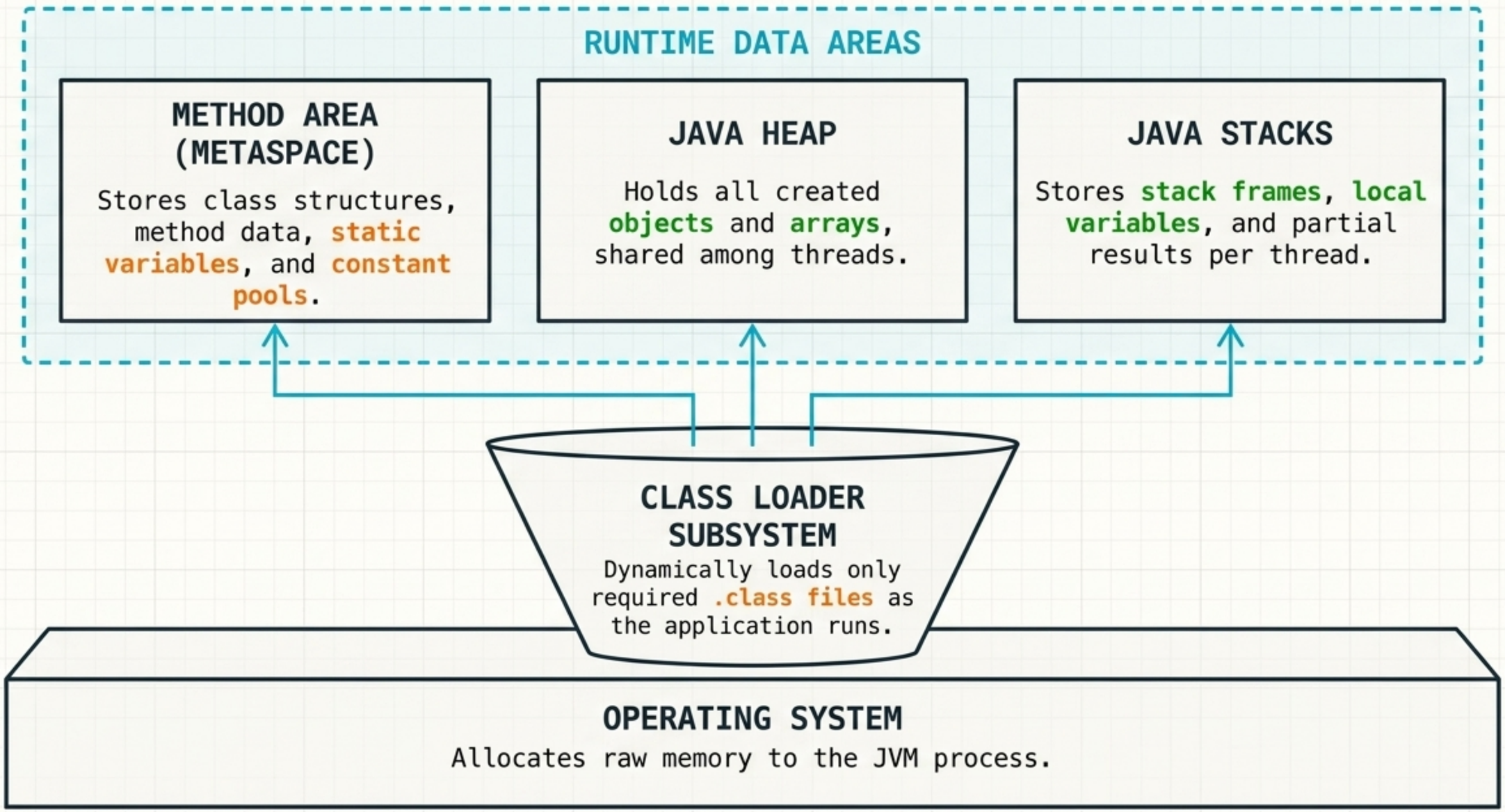
Arithmetic Promotion Rule

`byte + byte = int`

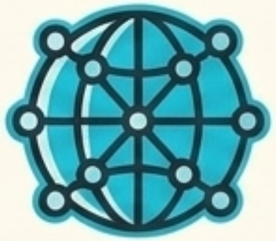
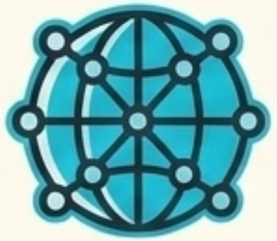
Any arithmetic operation involving **`byte`** or **`short`** operands automatically promotes the result to **`int`** to prevent overflow.

```
byte b3 = b1 + b2; // TRIGGERS COMPILE ERROR
byte b3 = (byte) (b1 + b2); // CORRECT
```

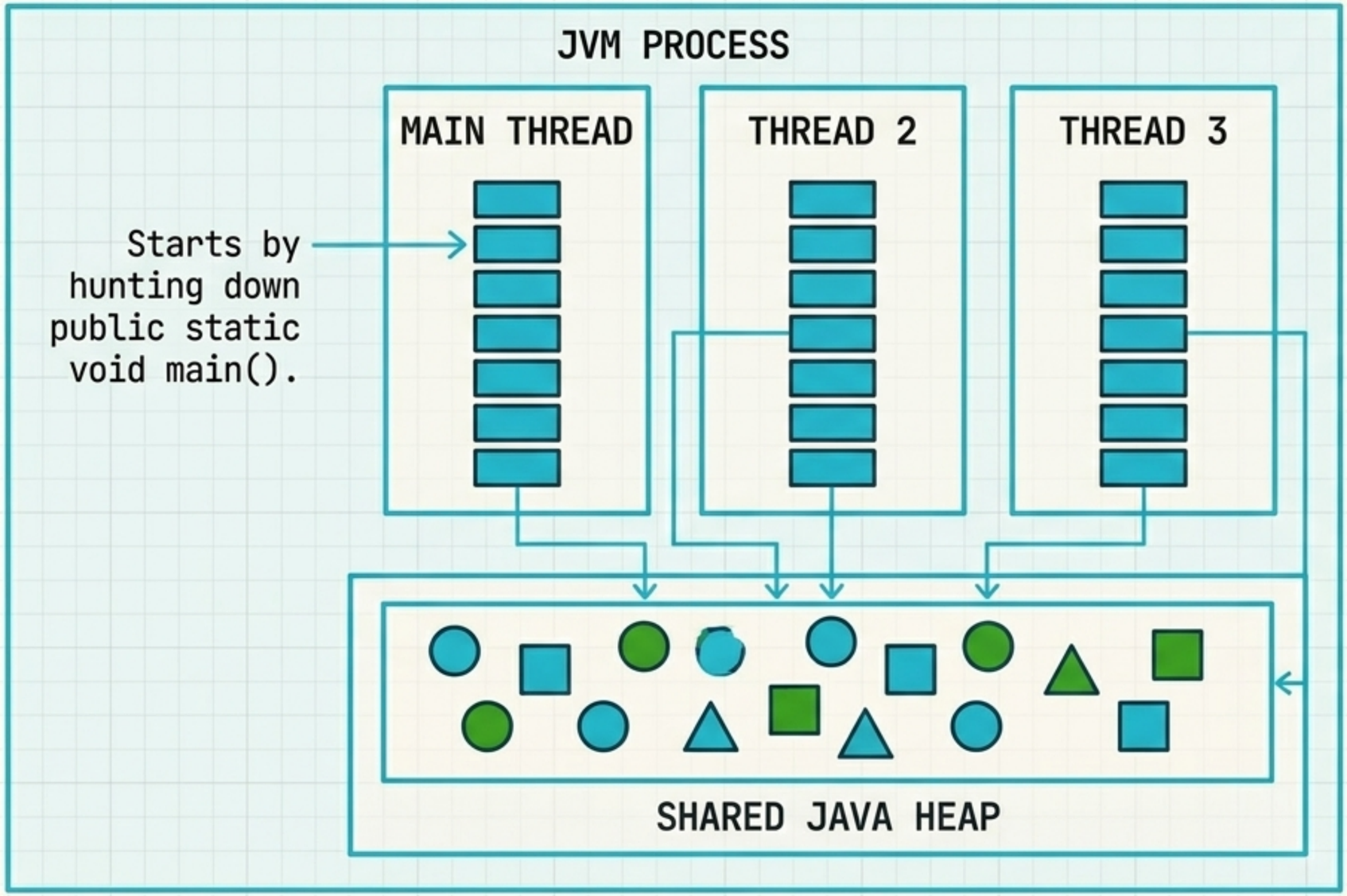
Java Virtual Machine (JVM) Architecture



JAVA MEMORY AREA COMPARISON

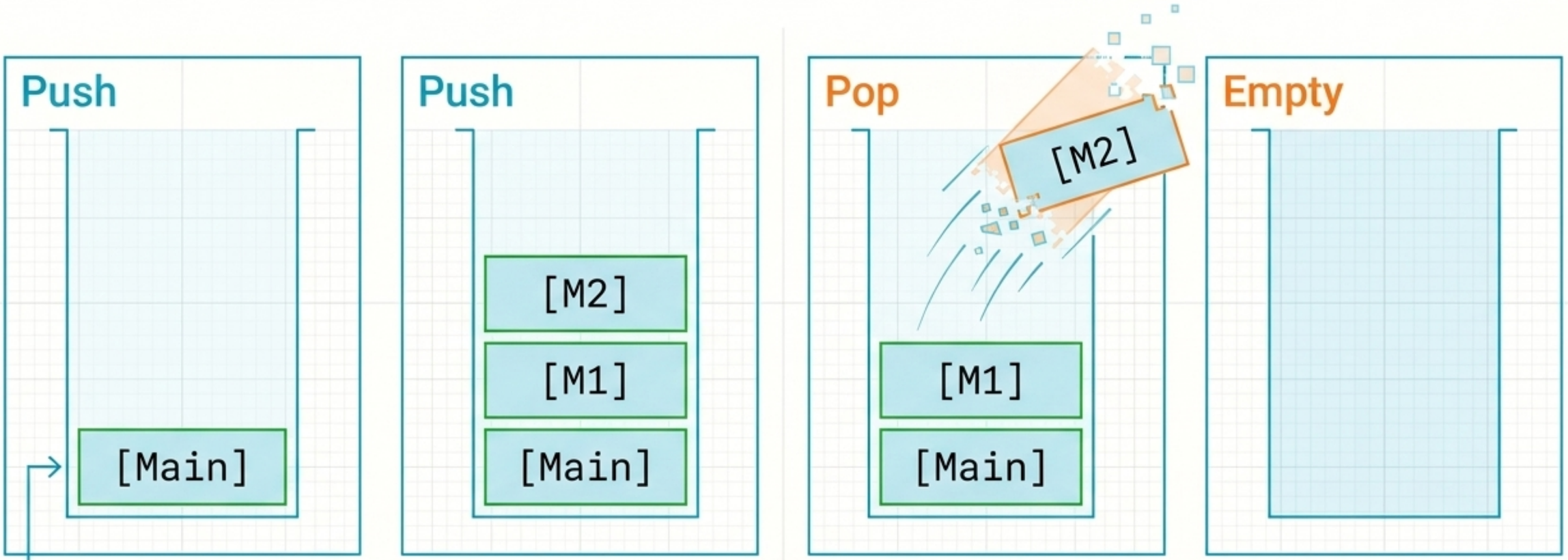
	METHOD AREA (METASPACE)	JAVA HEAP	JAVA STACKS
			
Scope	1 per JVM (Shared across all threads)	1 per JVM (Shared across all threads)	1 per Thread (Isolated memory)
Contents	Loaded `.class` structural info, `byte code`, and `static data members`.	Dynamically created `Java objects` (state/instance variables) and `Arrays`.	`Method local variables`, `method arguments`, `return values`, and `primitive values`.

JVM THREAD MEMORY ISOLATION



Thread Isolation:
While all threads can point to objects in the **shared Heap**, their individual execution paths are **strictly isolated**.

LIFO (Last-In, First-Out) Execution



Main thread invokes `main()`.

`main()` calls `M1()`, which calls `M2()`. Stack fills.

`M2()` finishes and returns. Frame popped off, local variables destroyed.

All methods return. Main Thread Stack is empty. Application terminates.

Synthesis: The Lifecycle of a Variable

Waypoint 1:
`Lexical Layer` **(Syntax)**
Identifier `'count'` passes naming rules (starts with a letter, camelCase).

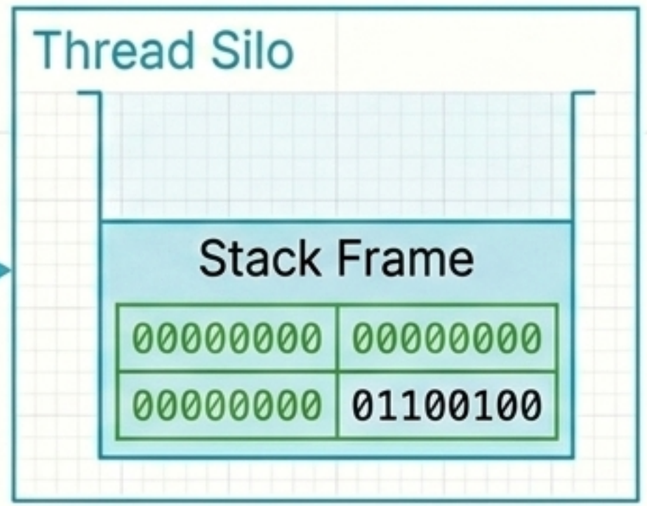
```
int count = 100;
```

Waypoint 2: **Compilation**
Compiler verifies `100` fits safely inside the integer range. No casting required.

Waypoint 3: **Physical Layer (Memory Shape)**

00000000	00000000
00000000	01100100

JVM preps a 4-byte block, storing `100` in binary using positive two's complement.



Waypoint 4:
Execution Layer (Runtime Location)
Pushed directly onto the active Stack Frame of the current thread. Popped and destroyed the moment the method returns.